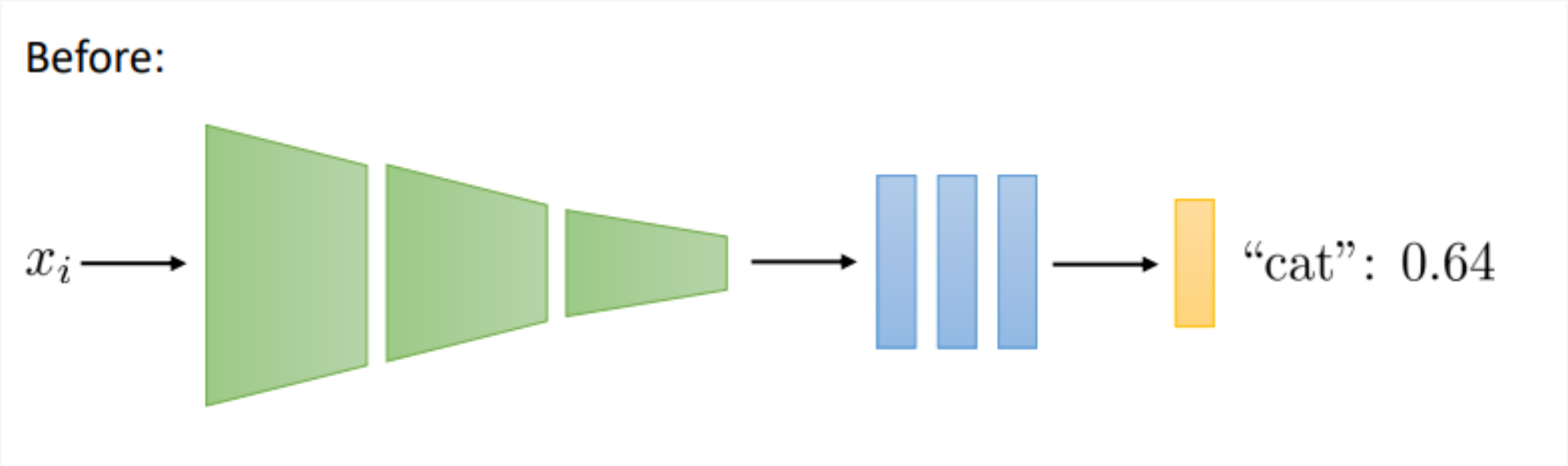
Sequence Modeling – Recurrent Neural Networks

Francisco Giral

Sequence Modeling

In the examples we have considered so far, we make predictions assuming each input output pair $x^{(i)}, y^{(i)}$ is independent identically distributed (i.i.d.)

In practice, many cases where the input/output pairs are given in a specific *sequence*, and we need to use the information about this sequence to help us make predictions

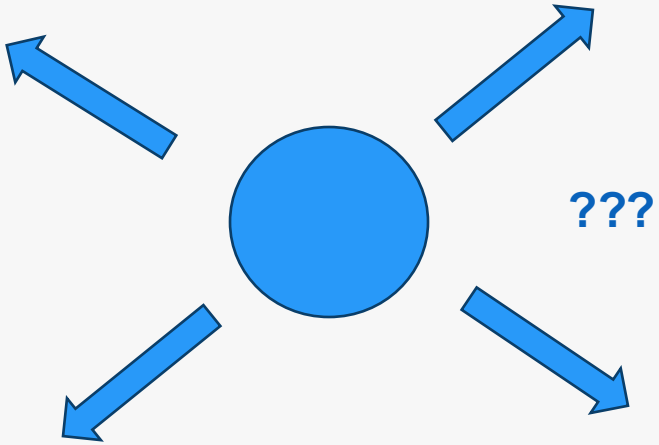


Sequence Modeling

Given an image of a ball, can you predict where it will go next?

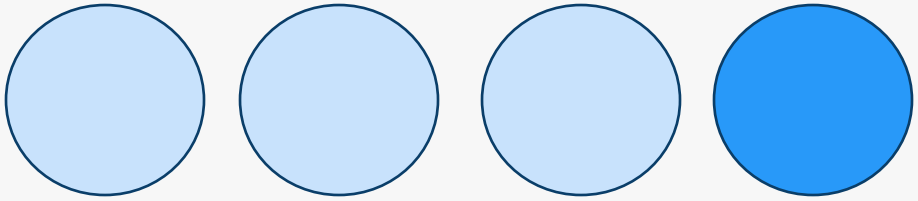
Sequence Modeling

Given an image of a ball, can you predict where it will go next?



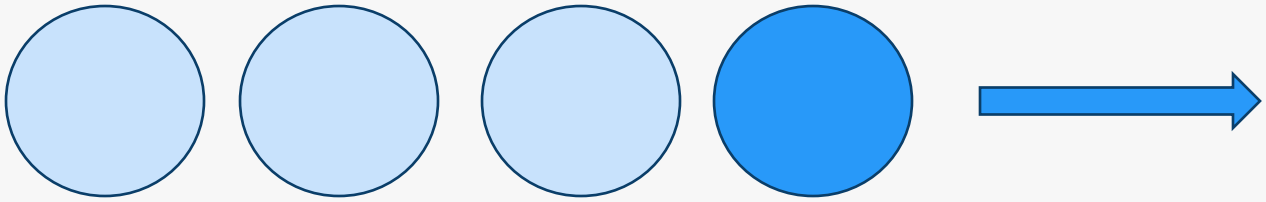
Sequence Modeling

Given an image of a ball, can you predict where it will go next?

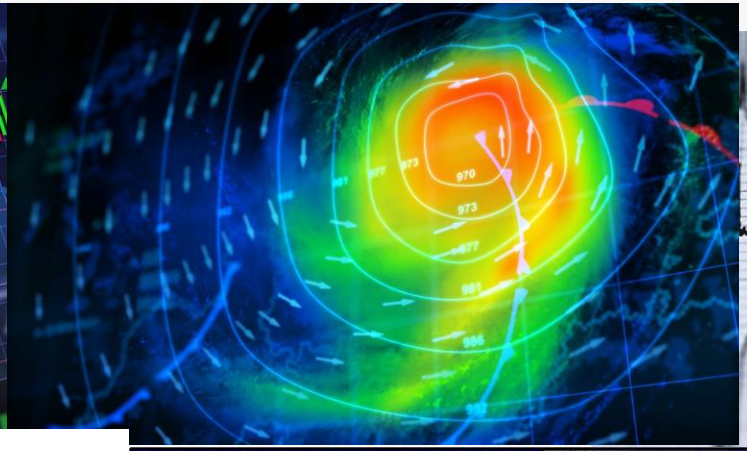


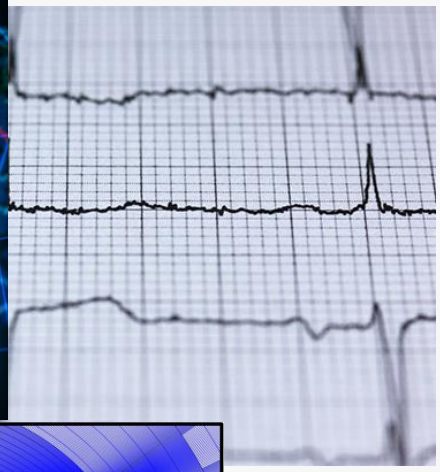
Sequence Modeling

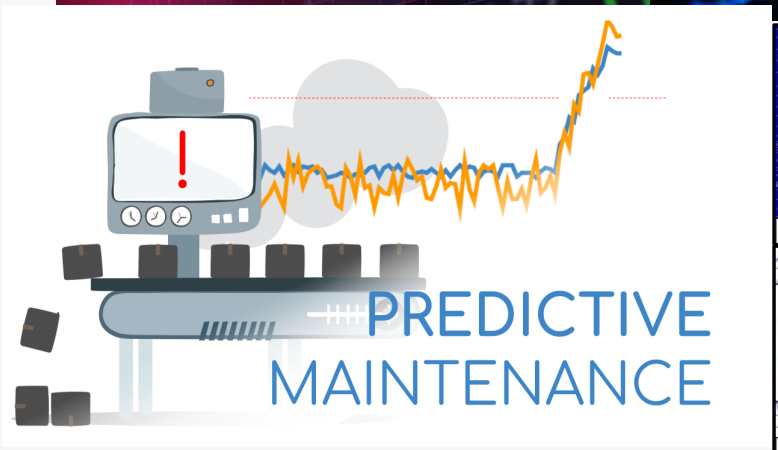
Given an image of a ball, can you predict where it will go next?

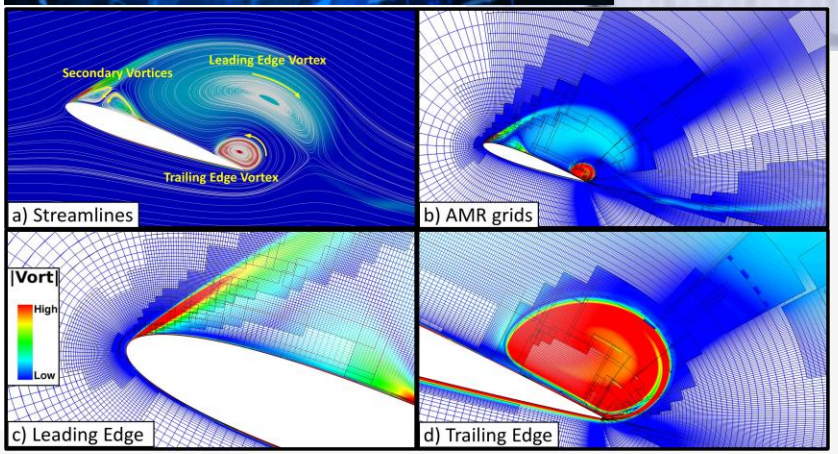


Sequence Modeling

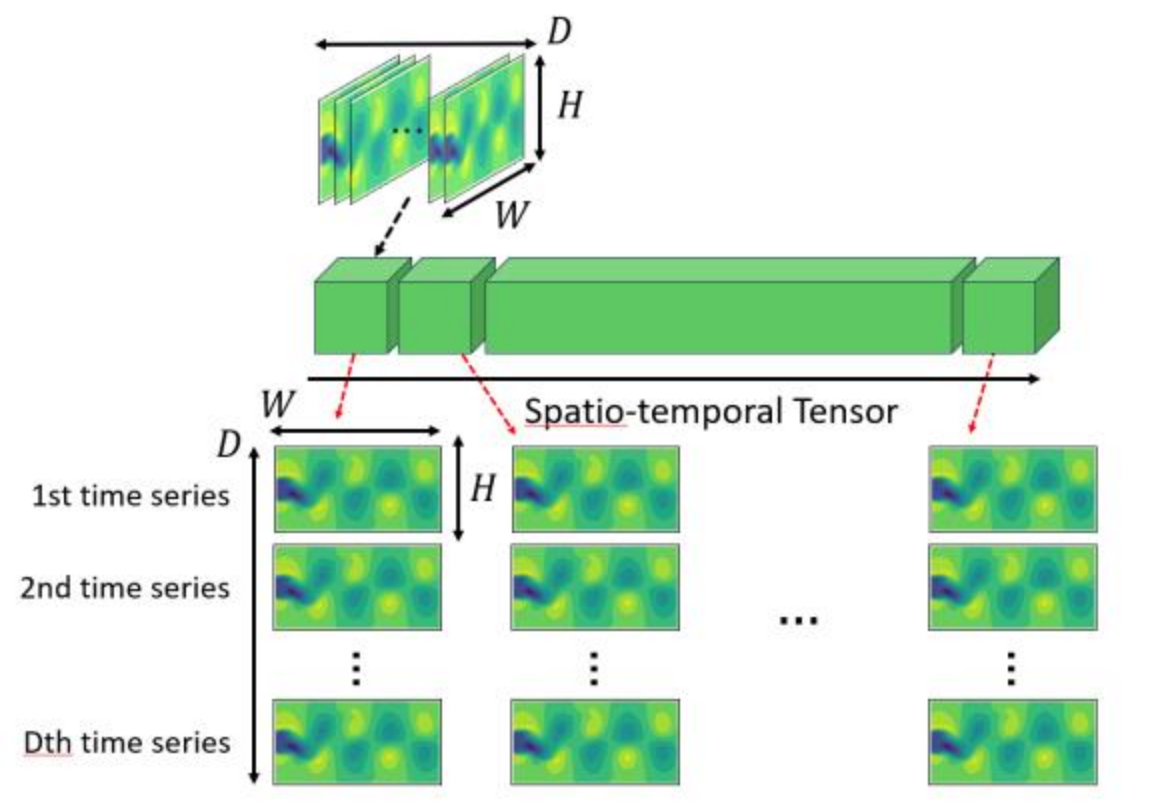






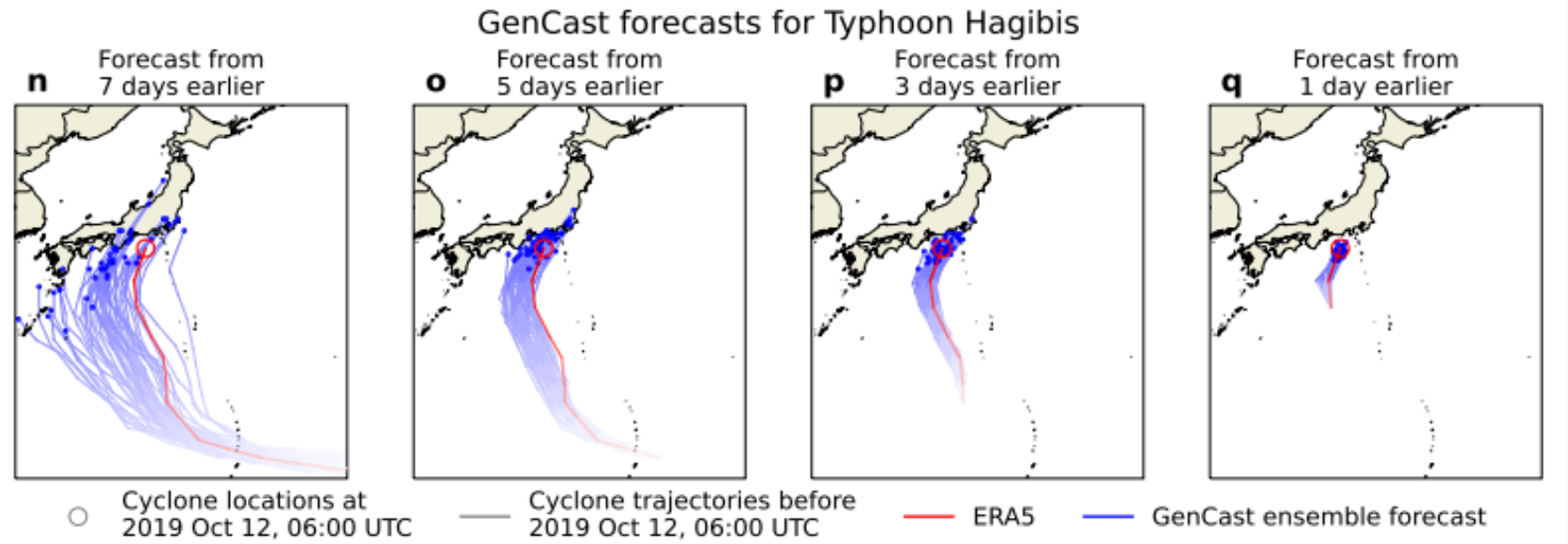


Example: Spatio-Temporal Problems in Fluid Dynamics



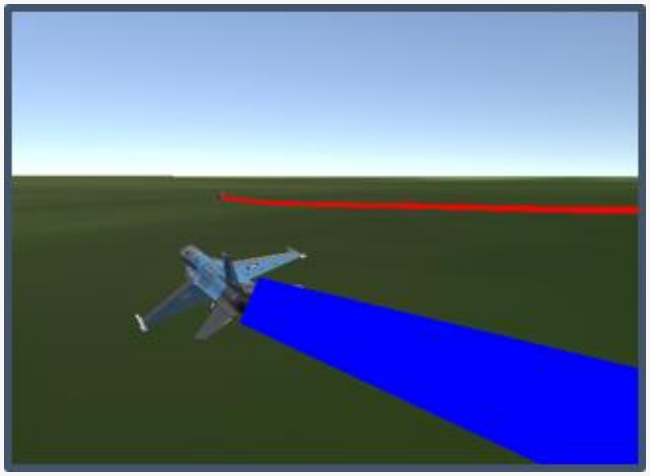
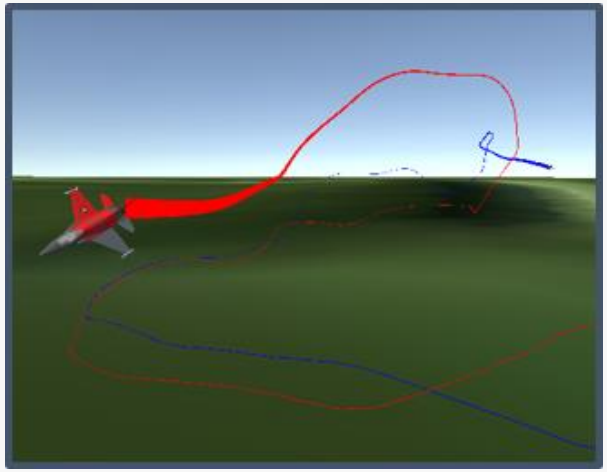
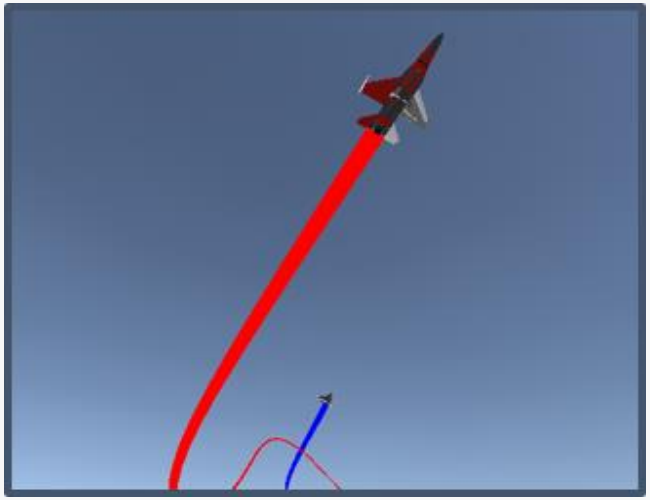
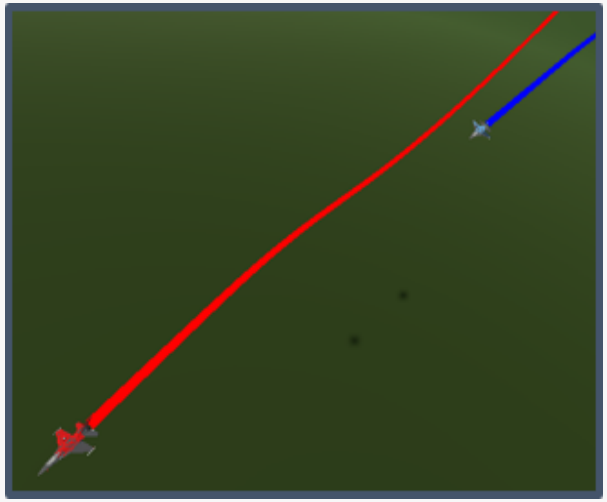
Abadía-Heredia, Rodrigo, et al. "Generalization capabilities and robustness of hybrid machine learning models grounded in flow physics compared to purely deep learning models." *arXiv preprint arXiv:2404.17884* (2024).

Example: Weather Forecasting



Price, Ilan, et al. "Gencast: Diffusion-based ensemble forecasting for medium-range weather." *arXiv preprint arXiv:2312.15796* (2023).

Example: Vehicle Trajectory Prediction

Example: Vehicle Trajectory Prediction

- ϕ: Latitude
- λ: Longitude
- h: Altitude
- ϕ: Roll angle
- θ: Pitch angle
- ψ: Yaw angle
- p: Roll rate
- q: Pitch rate
- r: Yaw rate

Example: Vehicle Trajectory Prediction

- ϕ: Latitude
- λ: Longitude
- h: Altitude
- ϕ: Roll angle
- θ: Pitch angle
- ψ: Yaw angle
- p: Roll rate
- q: Pitch rate
- r: Yaw rate

$$\mathbf{S}_{t-N:t} = \begin{bmatrix} \phi_{t-N} & \phi_{t-N+1} & \cdots & \phi_t \\ \lambda_{t-N} & \lambda_{t-N+1} & \cdots & \lambda_t \\ h_{t-N} & h_{t-N+1} & \cdots & h_t \\ \phi_{t-N} & \phi_{t-N+1} & \cdots & \phi_t \\ \theta_{t-N} & \theta_{t-N+1} & \cdots & \theta_t \\ \psi_{t-N} & \psi_{t-N+1} & \cdots & \psi_t \\ p_{t-N} & p_{t-N+1} & \cdots & p_t \\ q_{t-N} & q_{t-N+1} & \cdots & q_t \\ r_{t-N} & r_{t-N+1} & \cdots & r_t \end{bmatrix}^\top$$

Example: Vehicle Trajectory Prediction

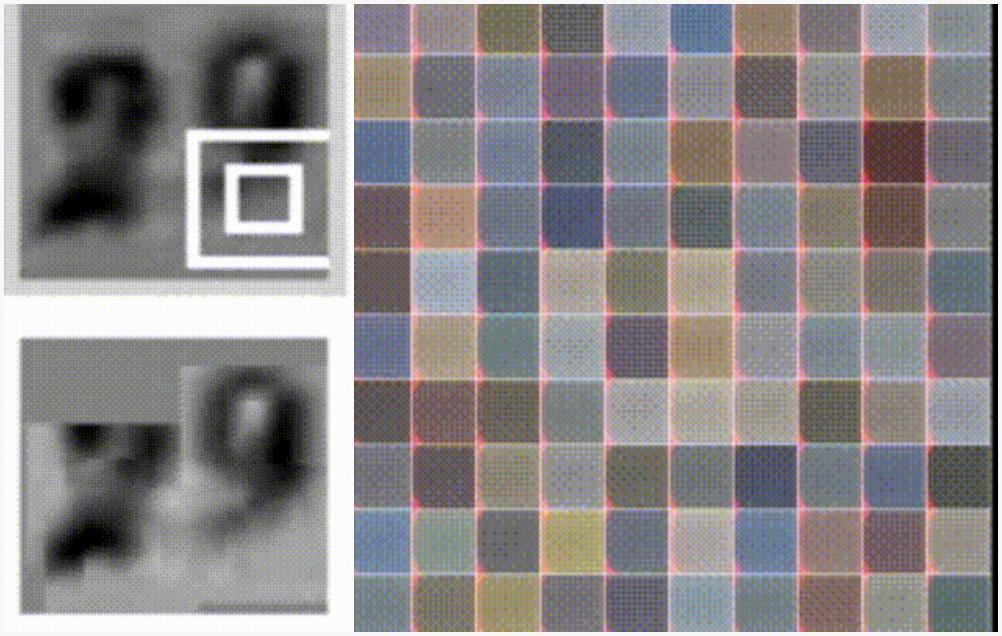
- ϕ: Latitude
- λ: Longitude
- h: Altitude
- ϕ: Roll angle
- θ: Pitch angle
- ψ: Yaw angle
- p: Roll rate
- q: Pitch rate
- r: Yaw rate

$$\mathbf{S}_{t-N:t} = \begin{bmatrix} \phi_{t-N} & \phi_{t-N+1} & \cdots & \phi_t \\ \lambda_{t-N} & \lambda_{t-N+1} & \cdots & \lambda_t \\ h_{t-N} & h_{t-N+1} & \cdots & h_t \\ \phi_{t-N} & \phi_{t-N+1} & \cdots & \phi_t \\ \theta_{t-N} & \theta_{t-N+1} & \cdots & \theta_t \\ \psi_{t-N} & \psi_{t-N+1} & \cdots & \psi_t \\ p_{t-N} & p_{t-N+1} & \cdots & p_t \\ q_{t-N} & q_{t-N+1} & \cdots & q_t \\ r_{t-N} & r_{t-N+1} & \cdots & r_t \end{bmatrix}^\top \longrightarrow \mathbf{S}_{t+1} = \begin{bmatrix} \phi_{t+1} \\ \lambda_{t+1} \\ h_{t+1} \\ \phi_{t+1} \\ \theta_{t+1} \\ \psi_{t+1} \\ p_{t+1} \\ q_{t+1} \\ r_{t+1} \end{bmatrix}$$

$$\mathbf{S}_{t+1} = f(\mathbf{S}_{t-N:t})$$

Sequence Modeling

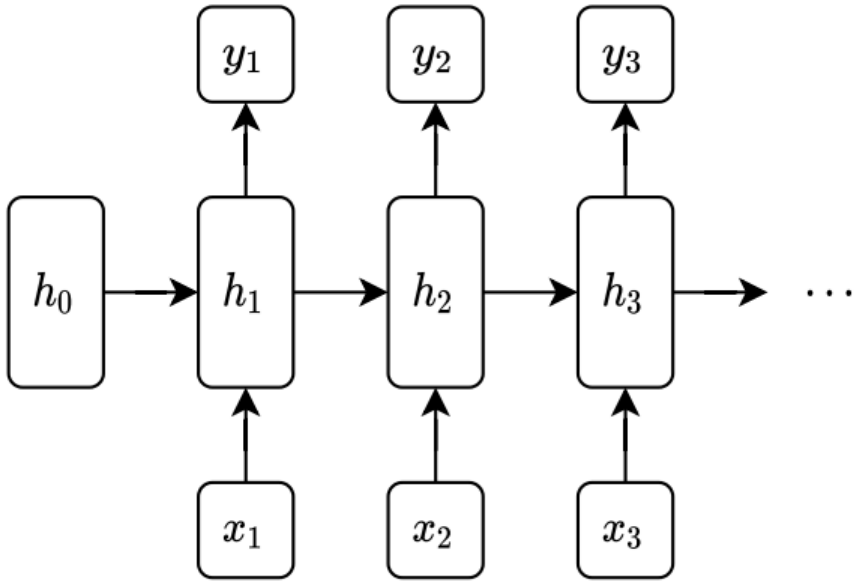
Many real world problems can be modeled as sequences, even the ones that do not seem like a sequence.



Left: RNN learns to read house numbers. Right: RNN learns to paint house numbers.

Recurrent Neural Networks

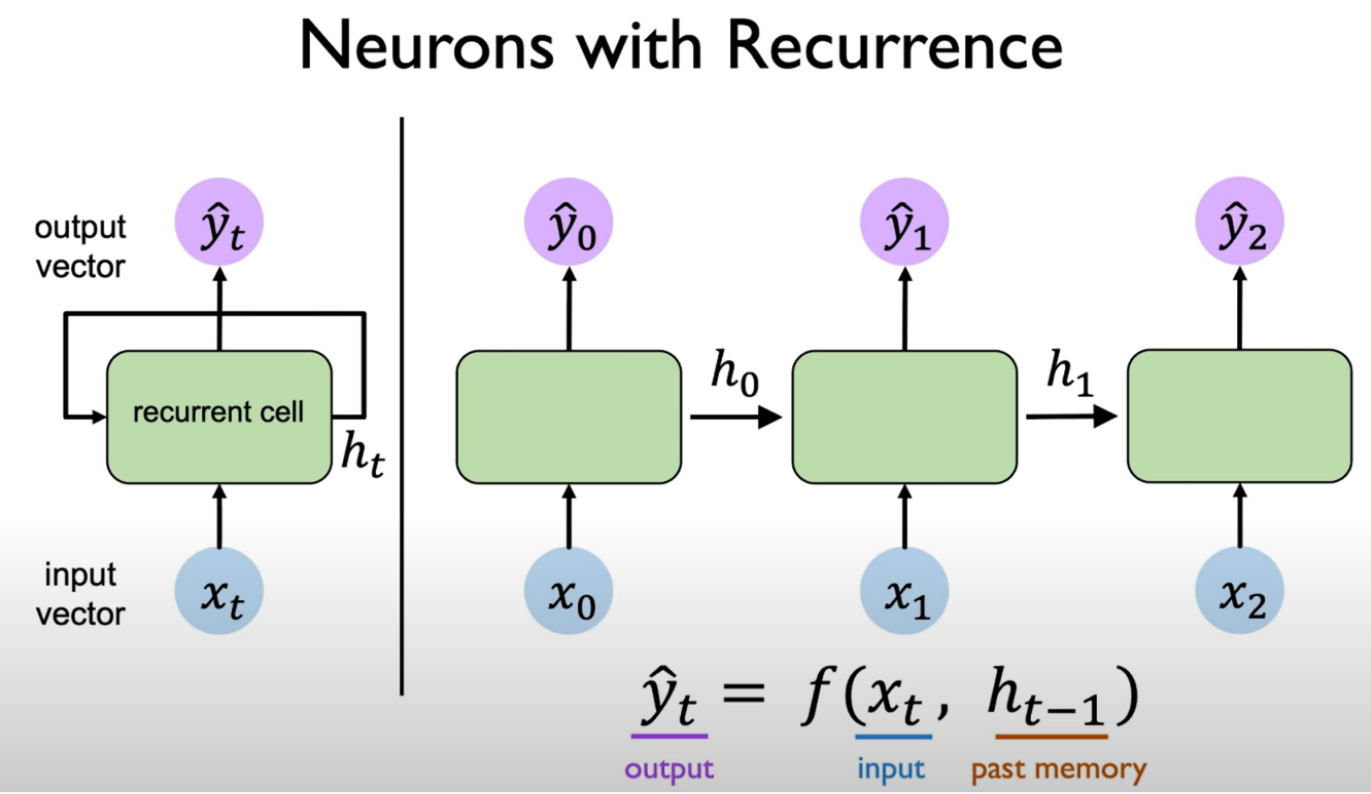
Recurrent neural networks (RNNs) maintain a hidden state over time, which is a function of the current input and previous hidden state



$$h_t = f(W_{hh}h_{t-1} + W_{hx}x_t + b_h)$$
$$y_t = g(W_{yh}h_t + b_y)$$

where f and g are activation functions, W_{hh}, W_{hx}, W_{yh} are weights and b_h, b_y are bias terms

Recurrent Neural Networks



Recurrent Neural Networks

Output Vector

$$\hat{y}_t = \boldsymbol{W}_{hy}^{T} h_t$$

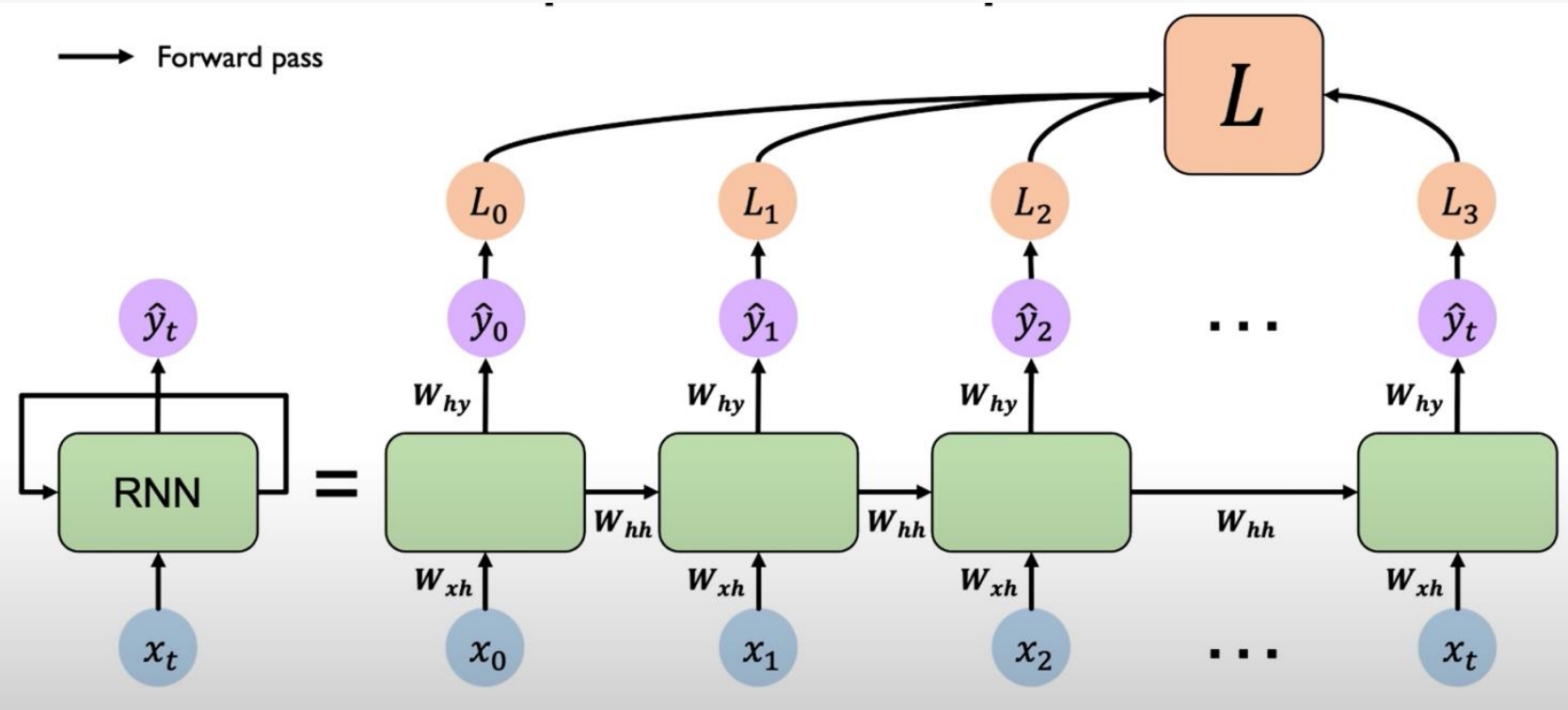
Update Hidden State

$$h_t = \tanh(\boldsymbol{W}_{hh}^{T} h_{t-1} + \boldsymbol{W}_{xh}^{T} x_t)$$

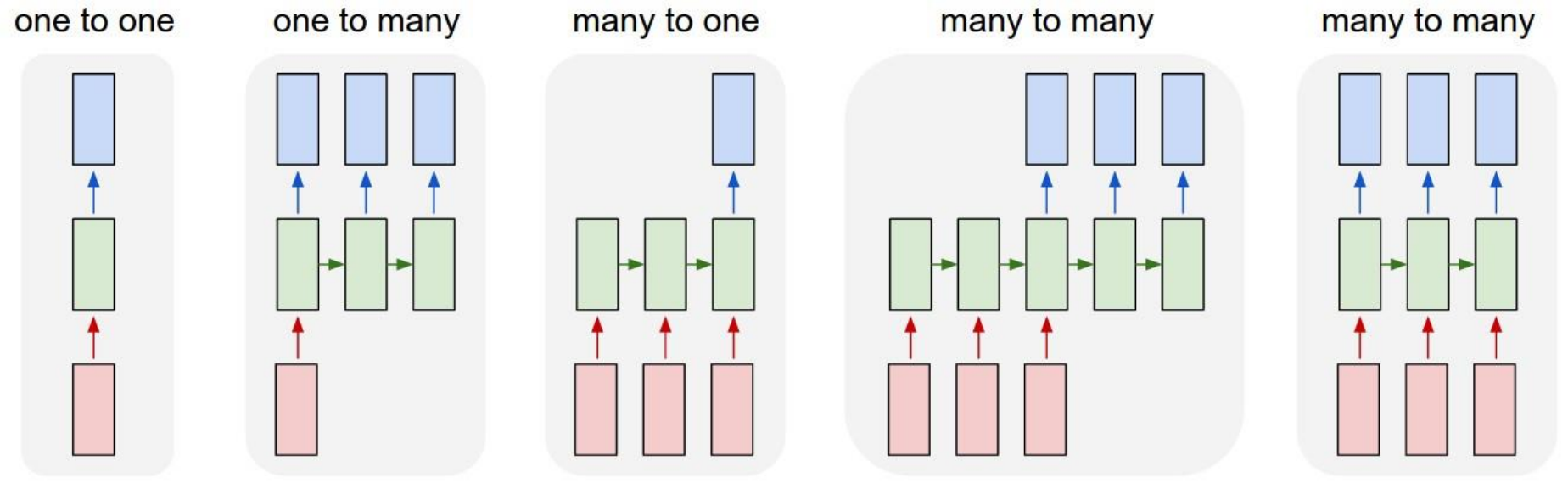
Input Vector

$$x_t$$

Recurrent Neural Networks

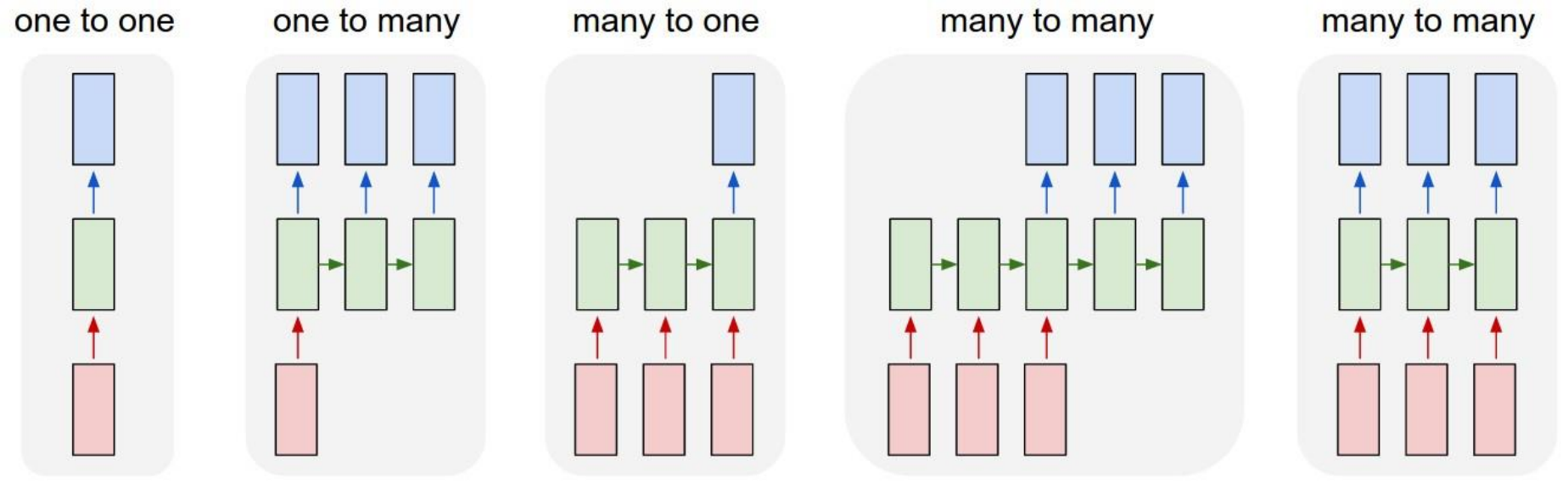


RNNs offer a lot of flexibility



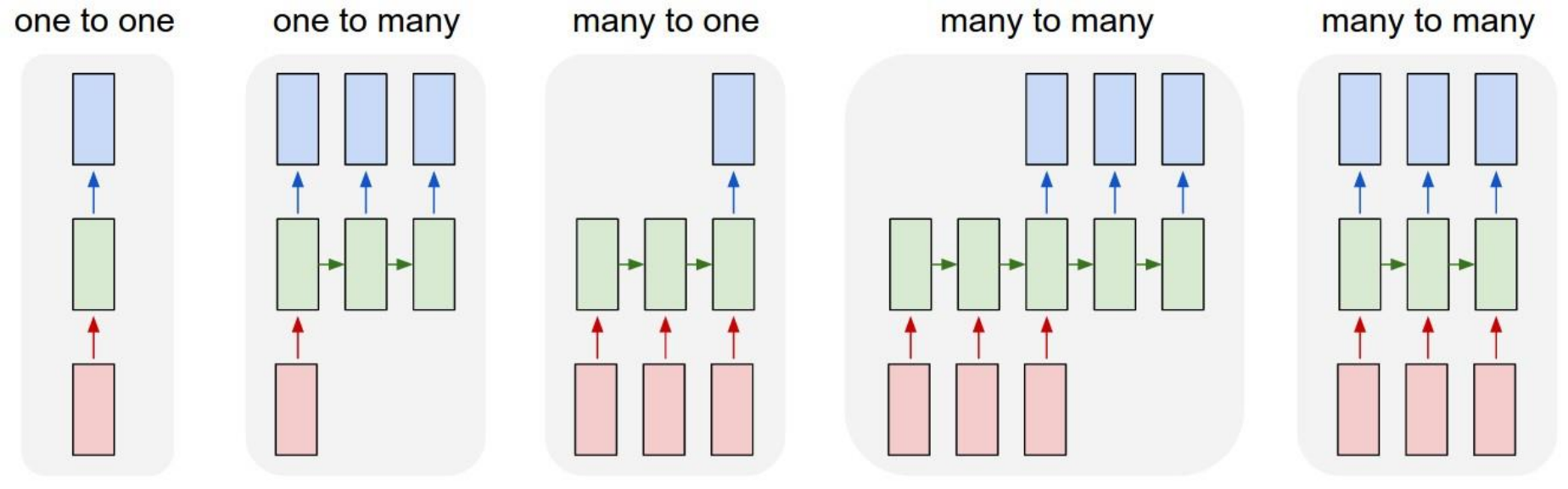
Vanilla Neural Networks

RNNs offer a lot of flexibility



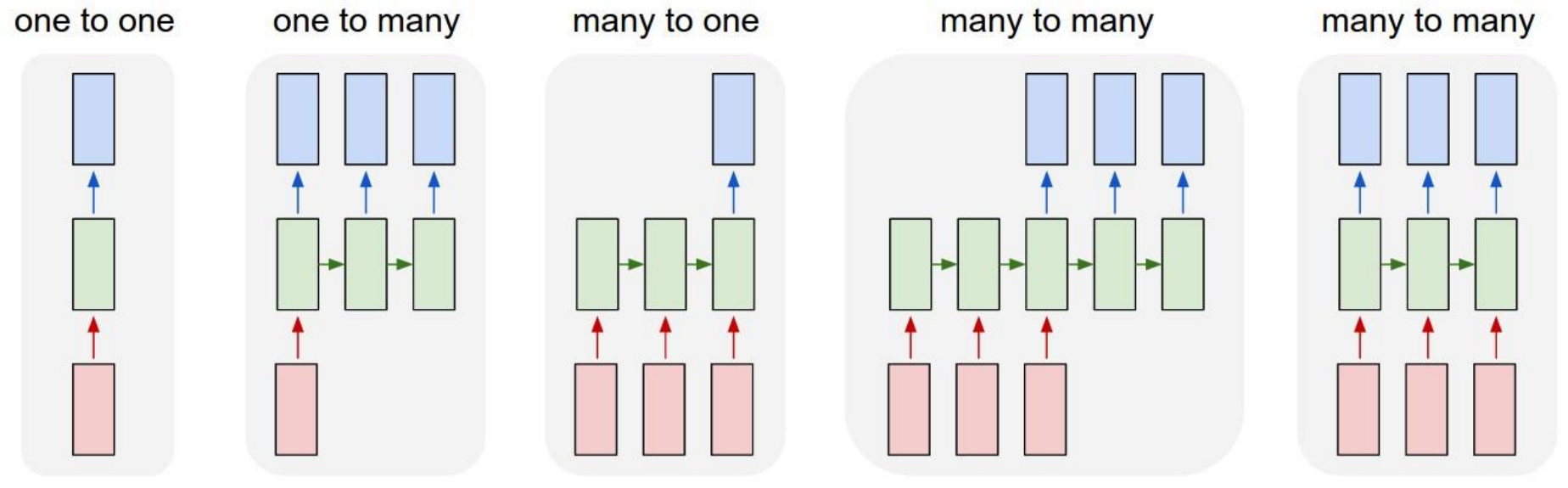
e.g. **Image Captioning**
image -> sequence of words

RNNs offer a lot of flexibility



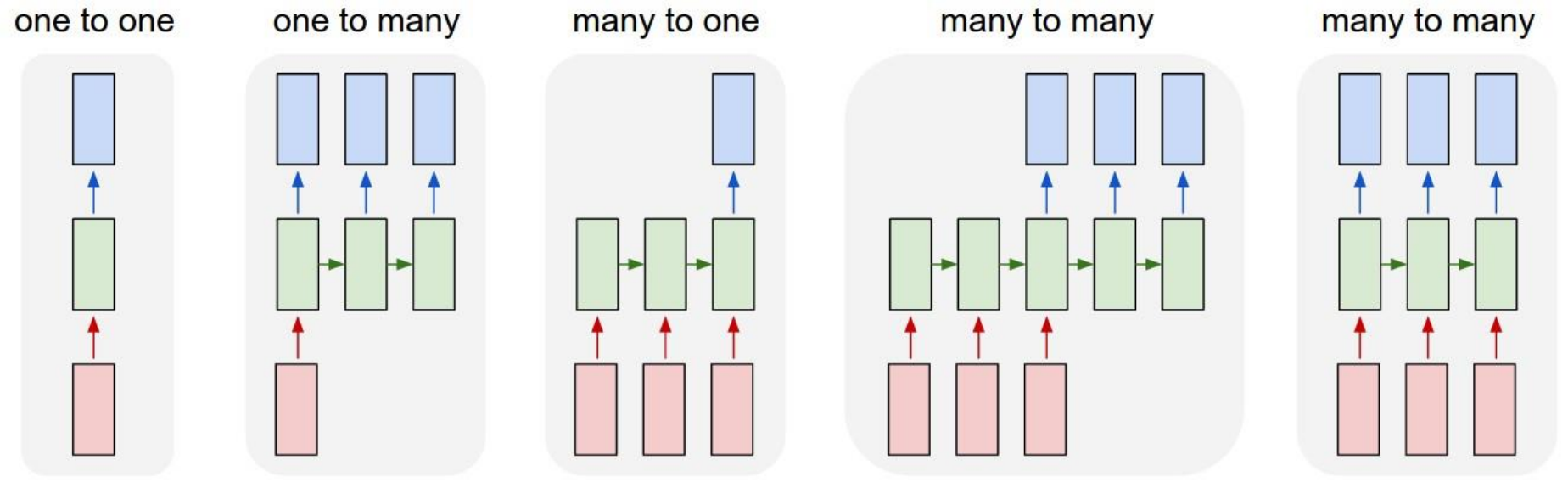
e.g. **Sentiment Classification**
sequence of words -> sentiment

RNNs offer a lot of flexibility



e.g. **Machine Translation**
seq of words -> seq of words

RNNs offer a lot of flexibility

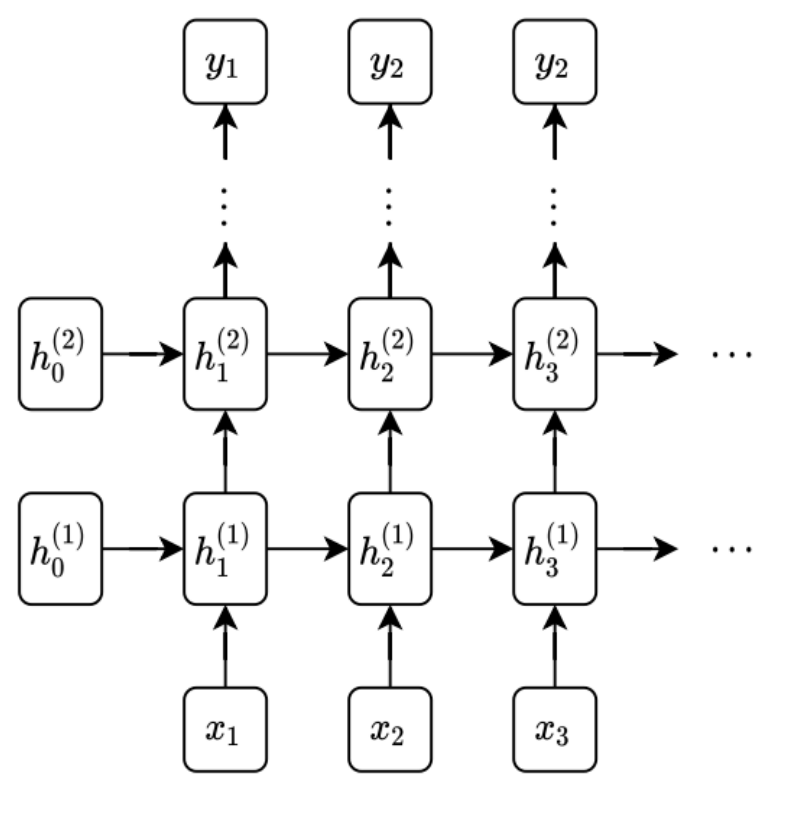


e.g. **Video classification** on frame level

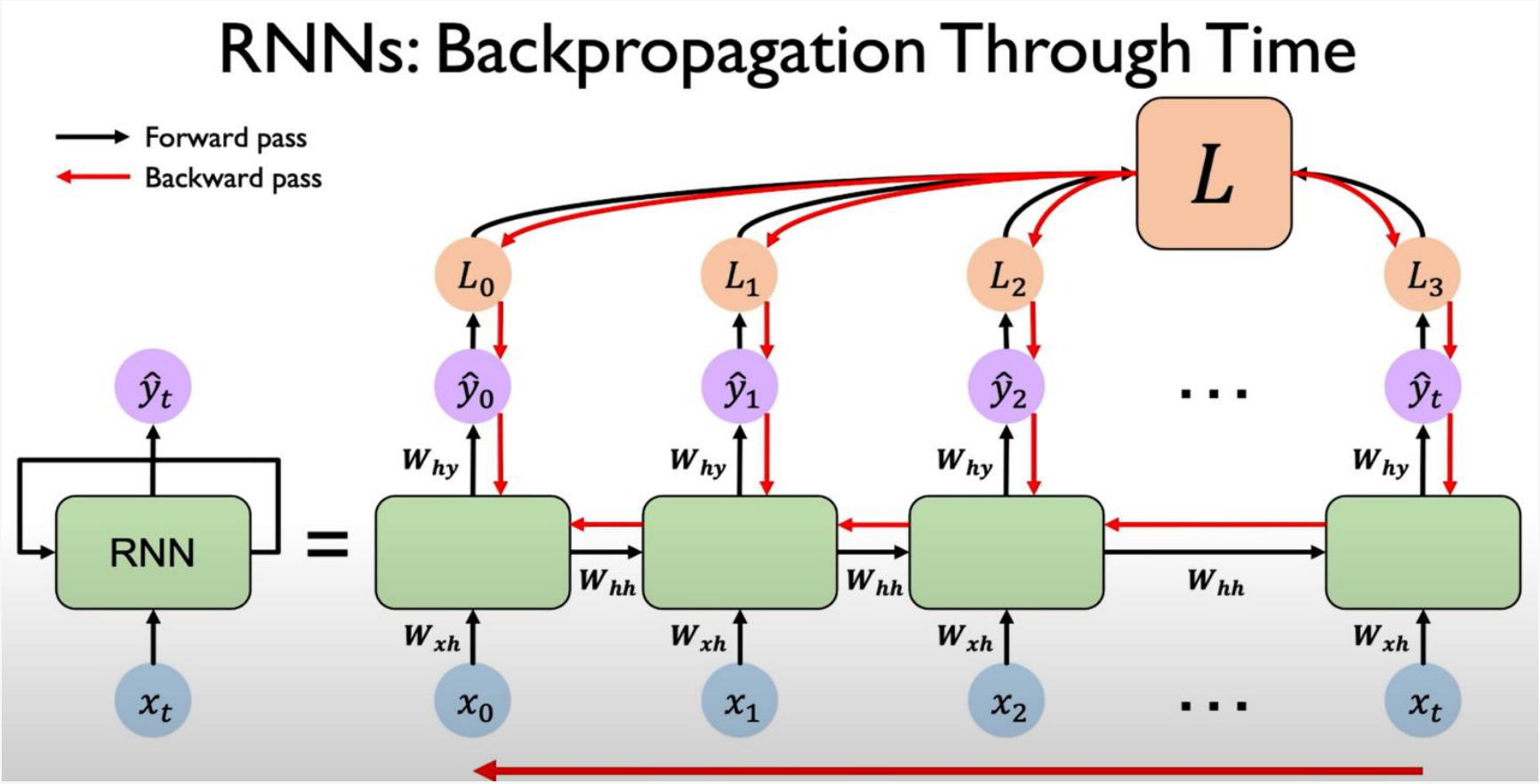
Stacking RNNs layers

Just like normal neural networks, RNNs can be stacked together, treating the hidden unit of one layer as the input to the next layer, to form “deep” RNNs

Practically speaking, tends to be less value in “very deep” RNNs than for other architectures



RNNs: Backpropagation Through Time



Challenges: Exploding/Vanishing Gradients

The challenge for training RNNs **is similar to that of training deep MLP networks.**

Because we train RNNs on long sequences, if the weights/activation of the RNN are scaled poorly, the hidden activations (and therefore also the gradients) will grow unboundedly with sequence length

Similarly, if weights are too small then information from the inputs will quickly decay with time (and it is precisely the “long range” dependencies that we would often like to model with sequence models)

imagine our sequence length was 1000+

that’s like backpropagating through 1000+ layers!

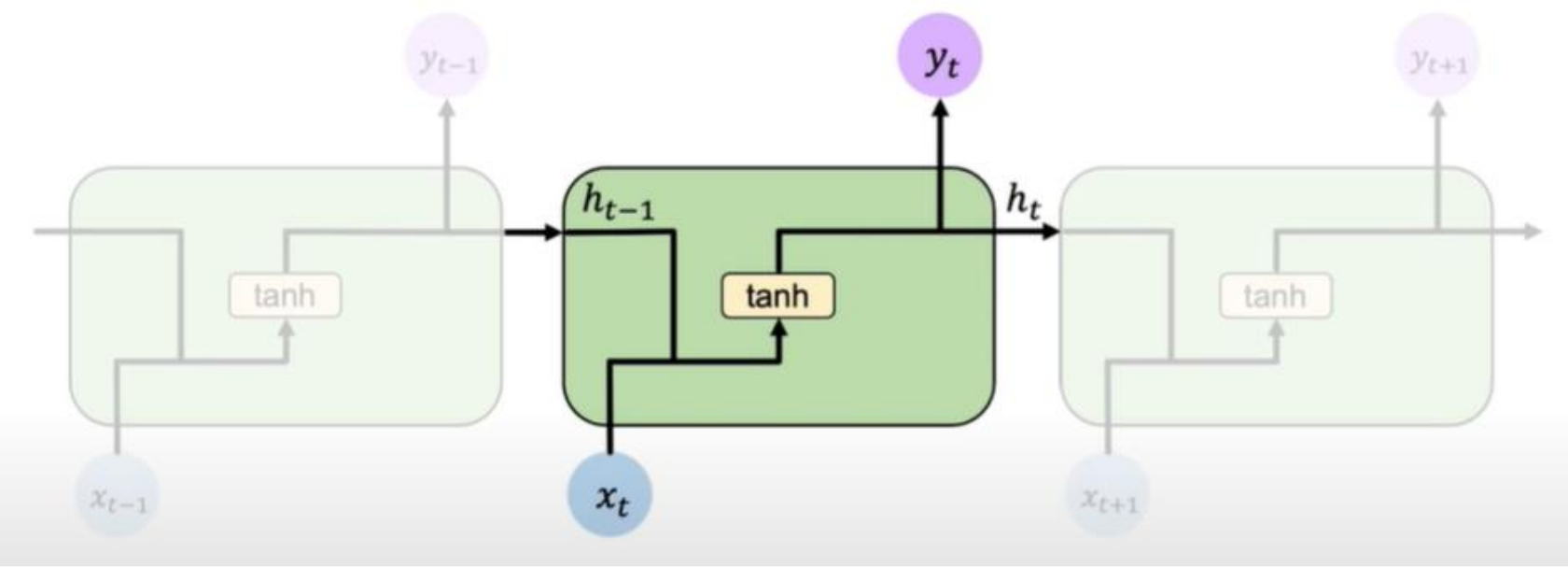
Alternatives to Vanilla RNN

Enhanced Architectures:

- Long Short Term Memory (LSTM)
- Gated Recurrent Units (GRU)

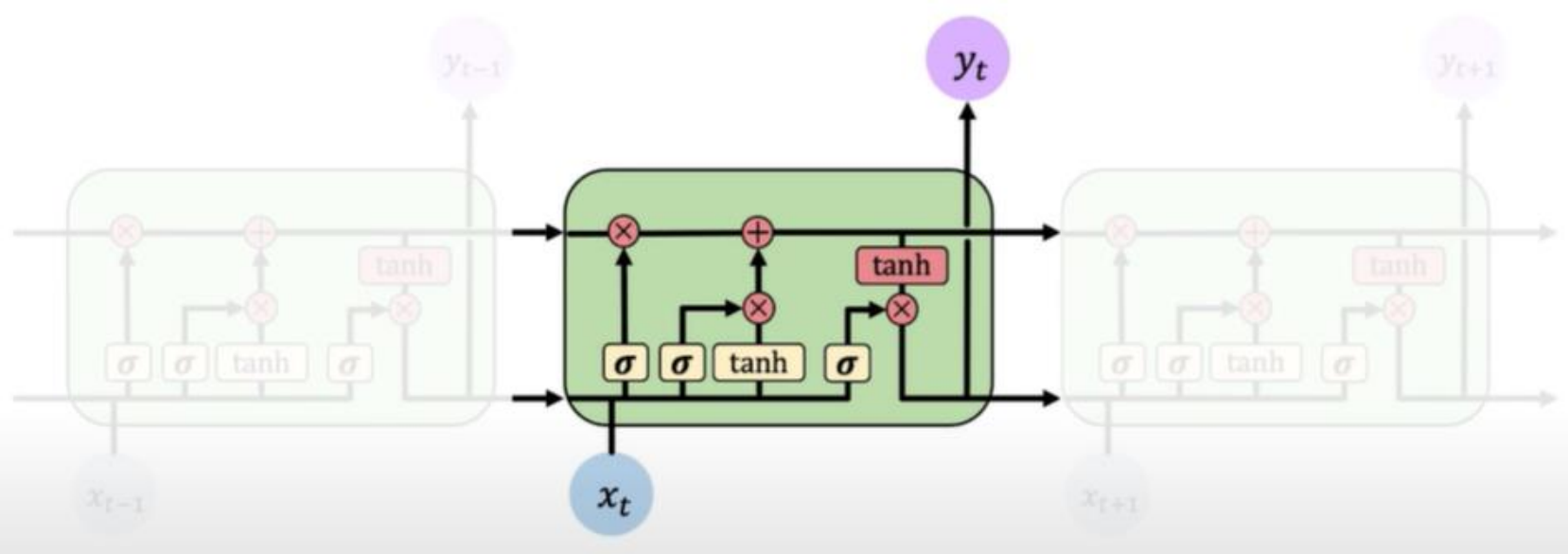
LSTMs

In a standard RNN, repeating modules contain a simple computation node.



LSTMs

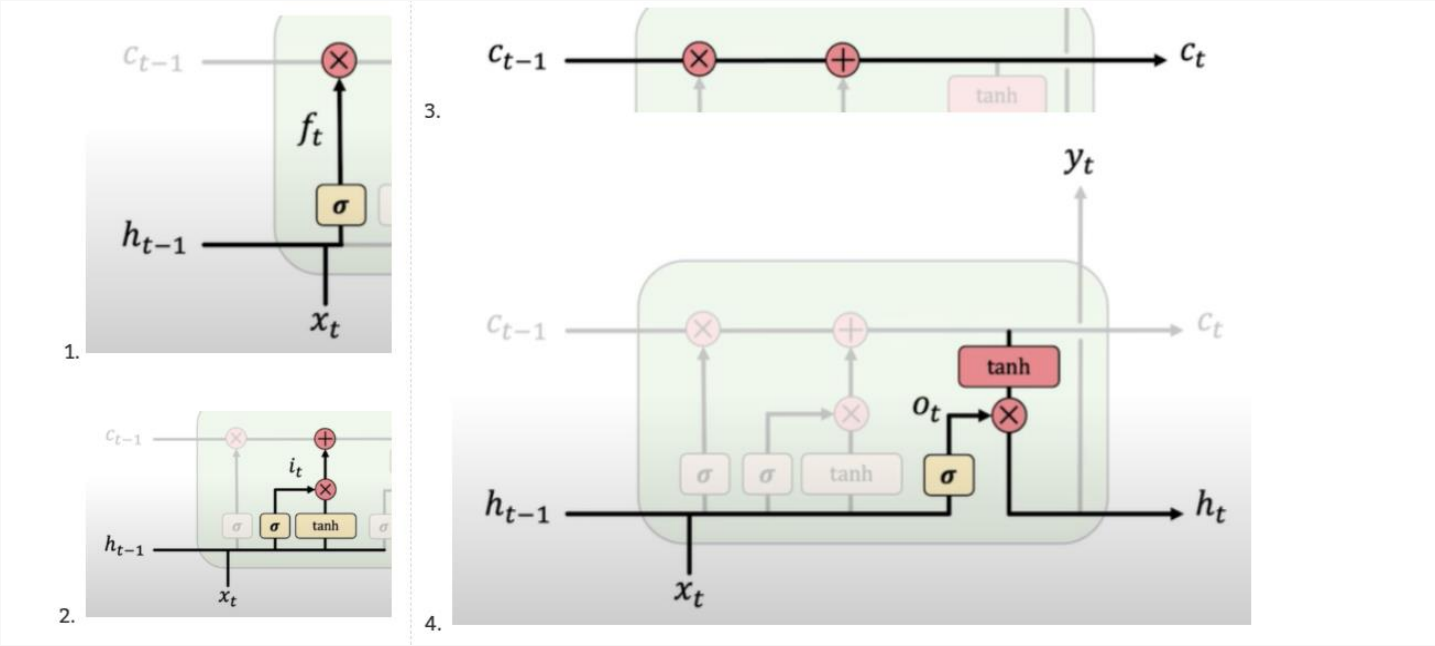
LSTM modules contain computational blocks that control information flow.



Information is added or removed through structures called gates. Gates optionally let information through, for example via a sigmoid neural net layer and pointwise multiplication.

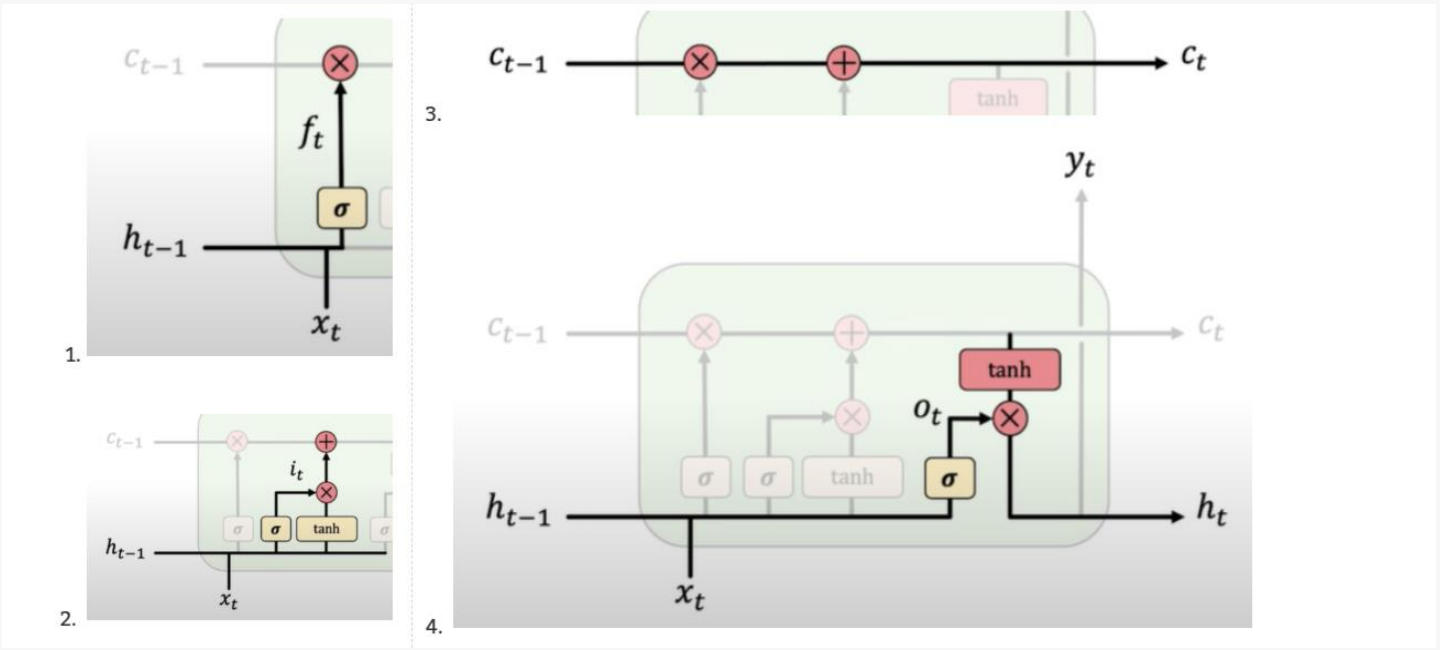
How do LSTMs work?

1. Forget. LSTMs forget irrelevant parts of the previous state.
2. Store. LSTMs store relevant new information into the cell state.
3. Update. LSTMs selectively update cell state values.
4. Output. The **output gate** controls what information is sent to the next time step.

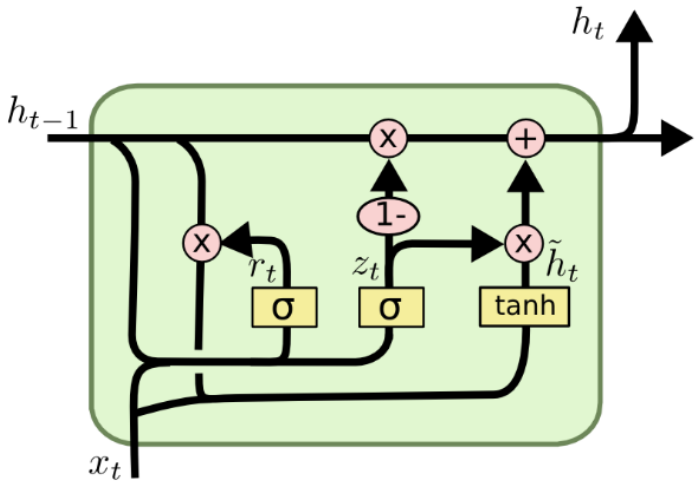


How do LSTMs work?

$$
\begin{aligned}
f_t &= \sigma_g(W_f x_t + U_f h_{t-1} + b_f) \\
i_t &= \sigma_g(W_i x_t + U_i h_{t-1} + b_i) \\
o_t &= \sigma_g(W_o x_t + U_o h_{t-1} + b_o) \\
c_t &= f_t \circ c_{t-1} + i_t \circ \sigma_c(W_c x_t + U_c h_{t-1} + b_c) \\
h_t &= o_t \circ \sigma_h(c_t)
\end{aligned}
$$



What about GRUs?



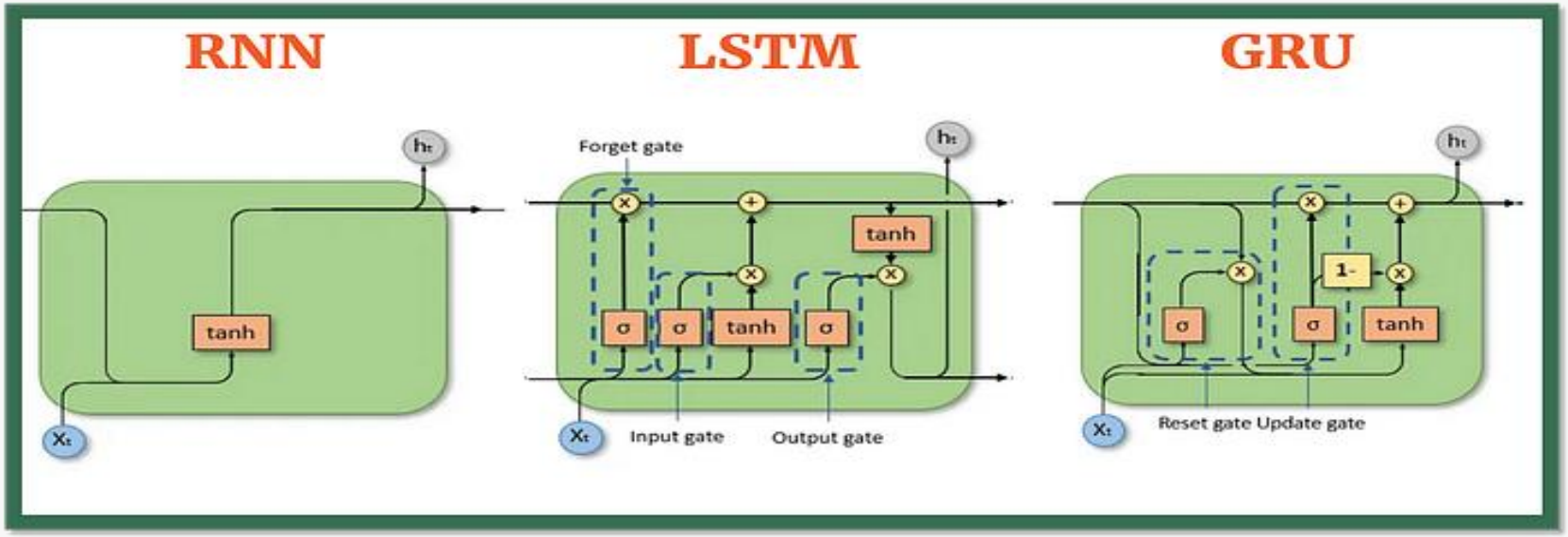
$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t]\right)$$

$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right)$$

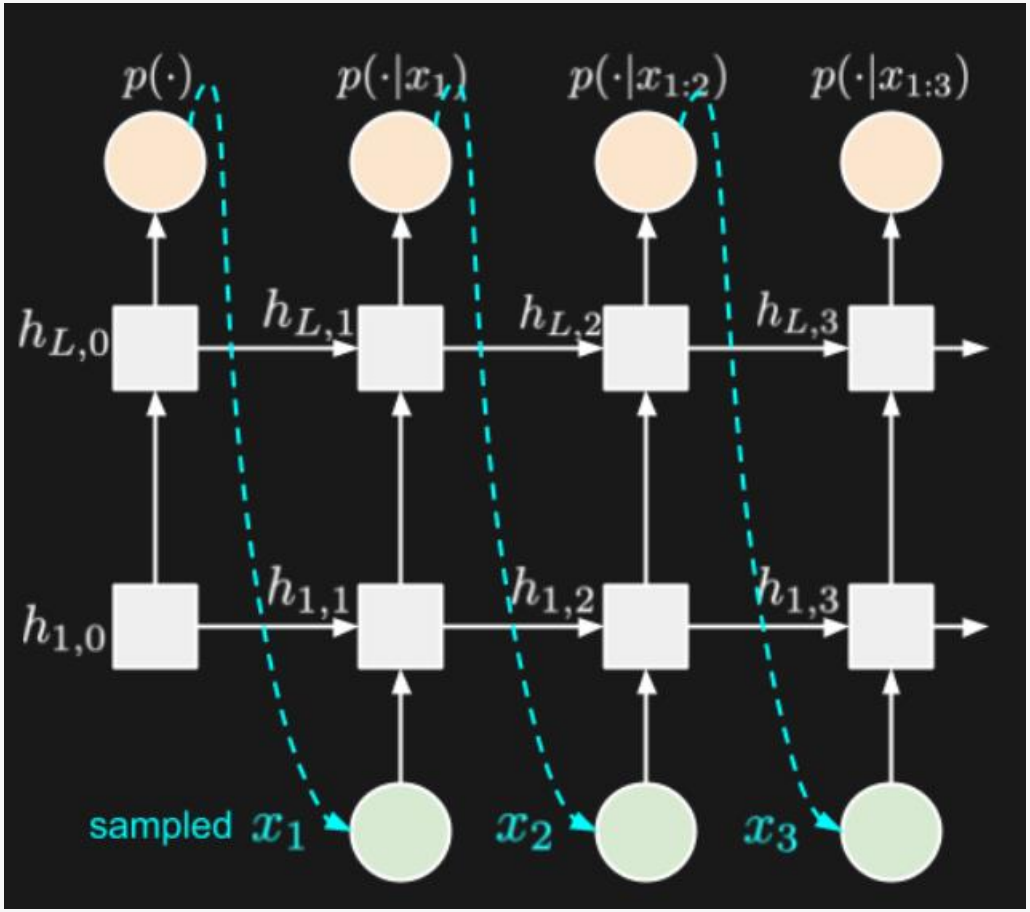
$$\tilde{h}_t = \tanh\left(W \cdot [r_t * h_{t-1}, x_t]\right)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

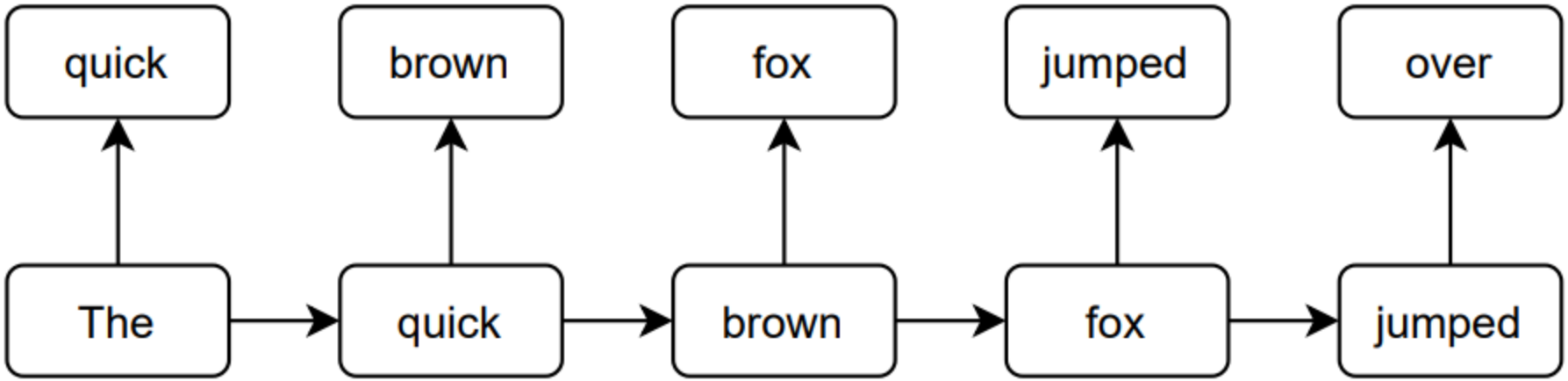
What about GRUs?



Using RNNs: Autoregressive Prediction



Example: Autoregressive Prediction

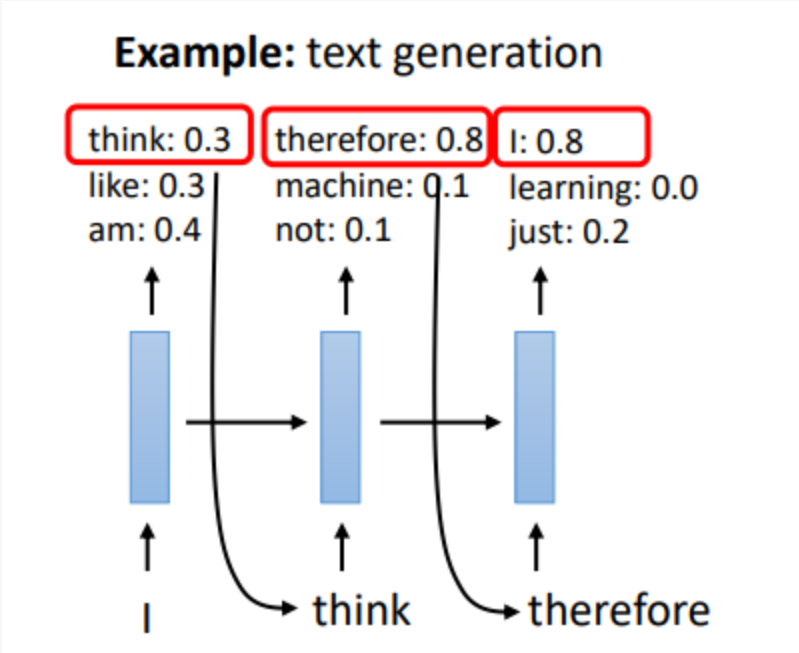


most RNNs used in practice look like this

why?

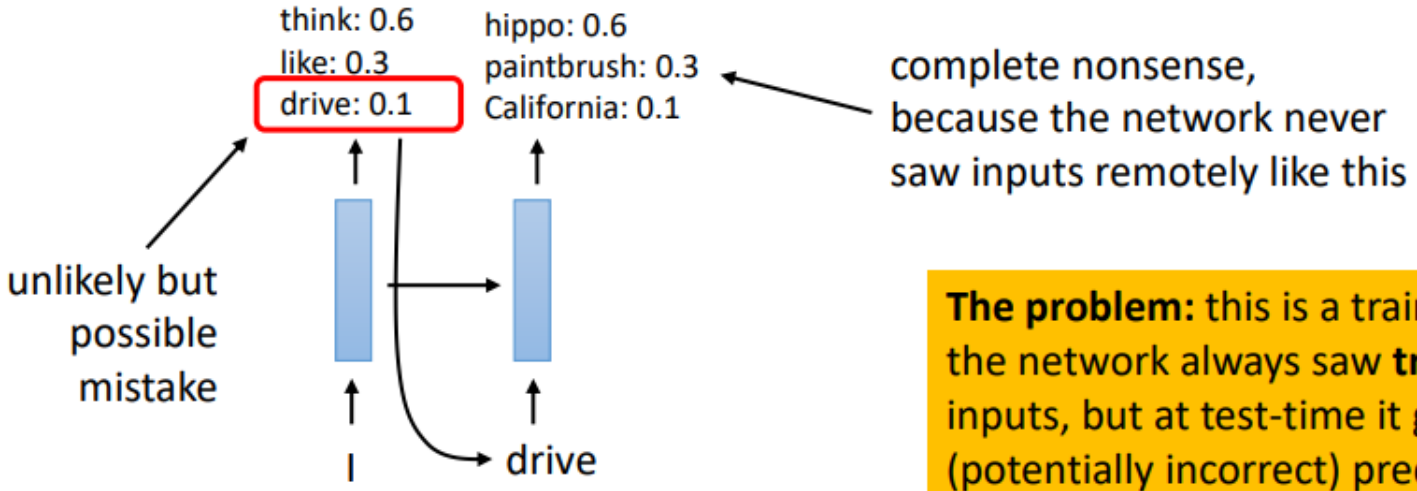
most problems that require multiple outputs have strong dependencies between these outputs

Example: Autoregressive Prediction



It is potentially easier for discrete use cases

Aside: distributional shift



we got unlucky, but now the model is completely confused

it never saw "I drive" before

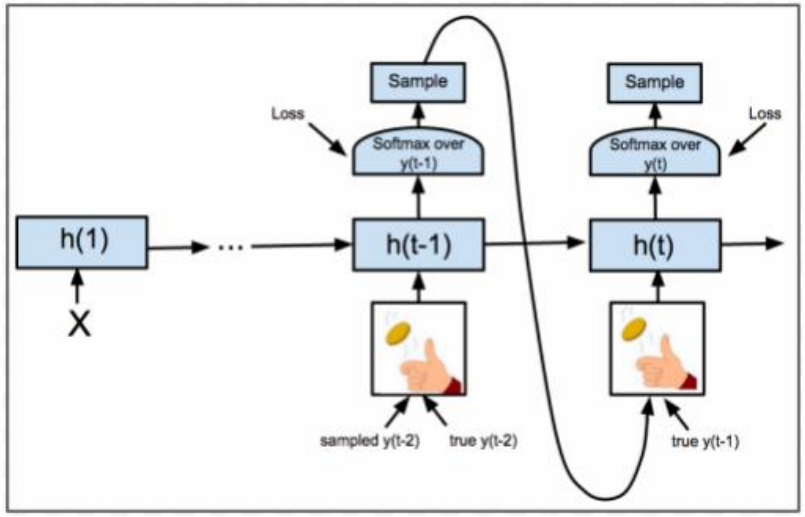
The problem: this is a training/test discrepancy: the network always saw **true** sequences as inputs, but at test-time it gets as input its own (potentially incorrect) predictions

This is called **distributional shift**, because the input distribution **shifts** from true strings (at training) to synthetic strings (at test time)

Even **one** random mistake can completely scramble the output!

Aside: scheduled sampling

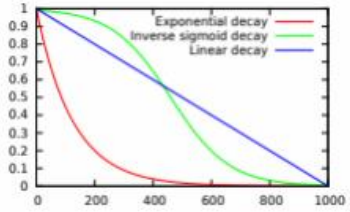
An old trick from reinforcement learning adapted to training RNNs



At the beginning of training, mostly feed in ground truth tokens as input, since model predictions are mostly nonsense

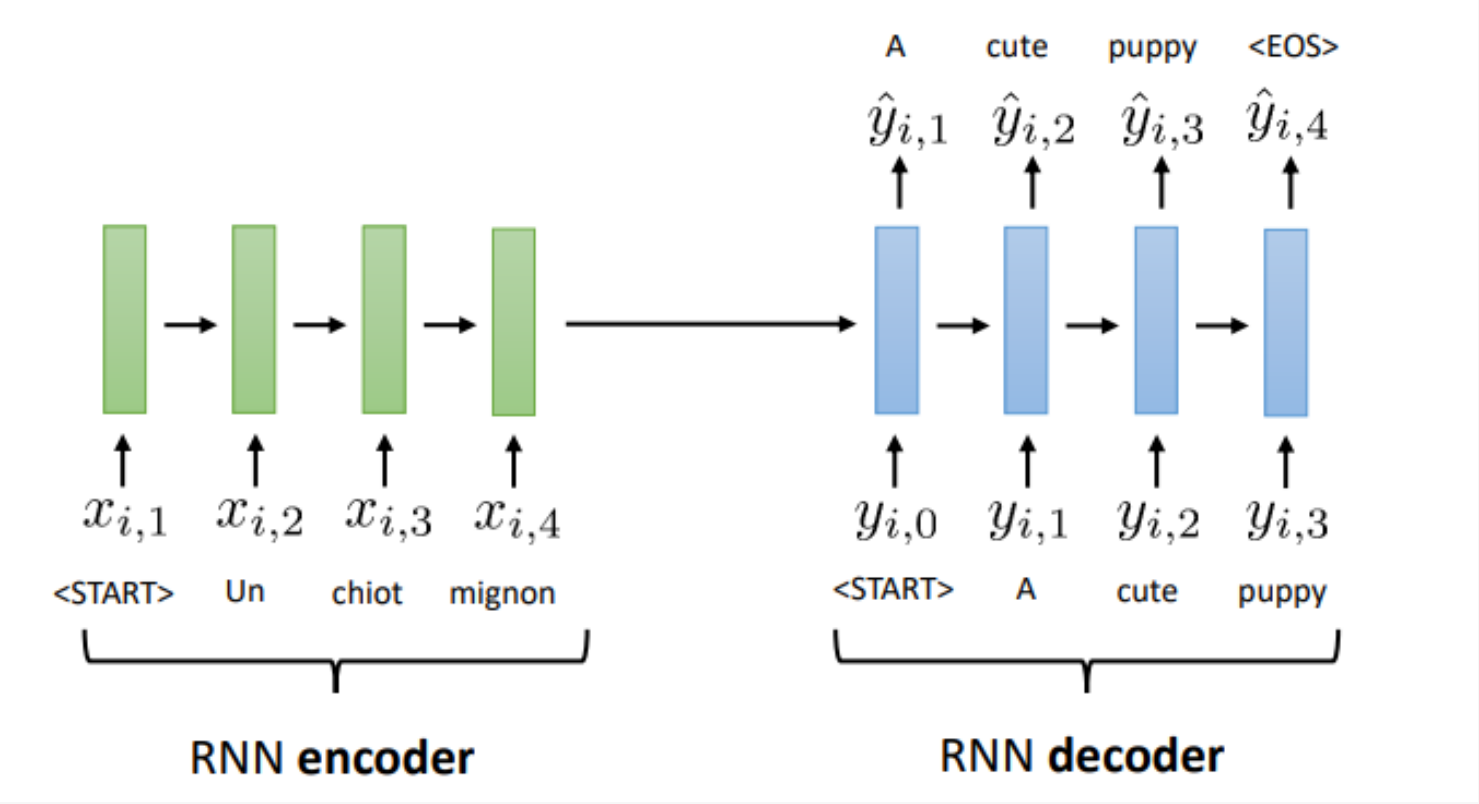
At the end of training, mostly feed in the model's own predictions, to mitigate distribution shift

schedules for probability of using ground truth input token

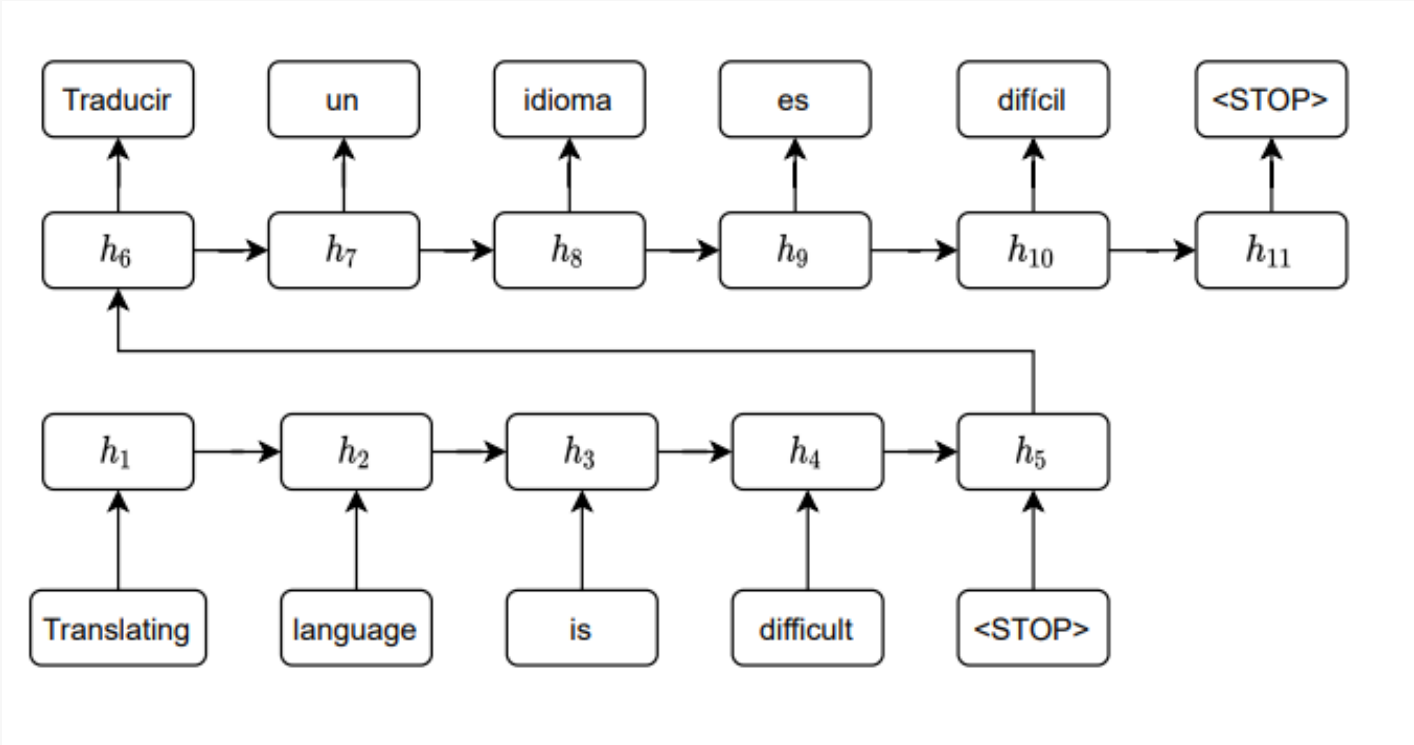


Randomly decide whether to give the network a ground truth token as input during training, or its own previous prediction

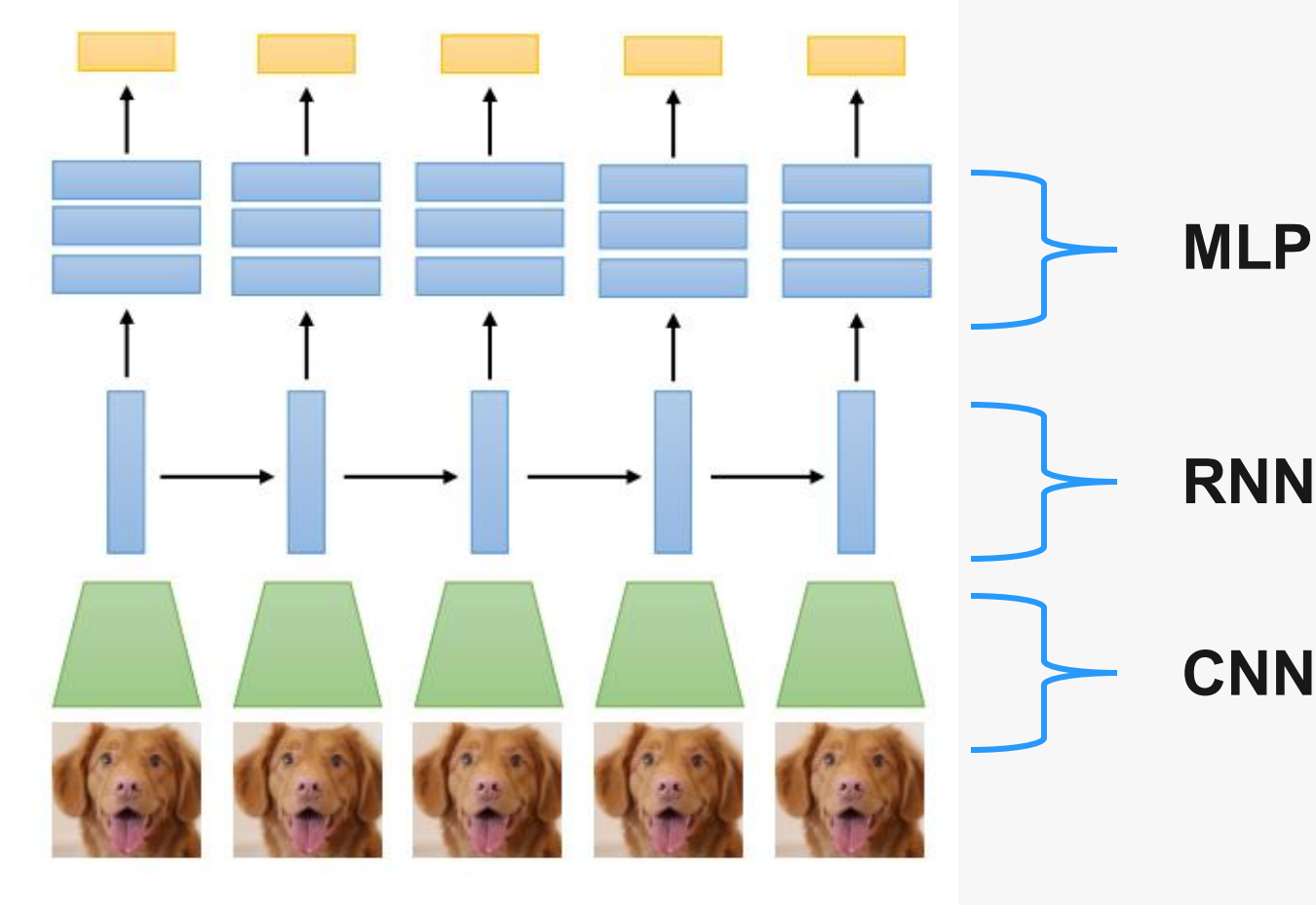
Beyond "simple" RNNs: Sequence-to-Sequence Models



Beyond Simple RNNs: Sequence-to-Sequence Models



Combination of Architectures



Practical Example: Aircraft Trajectory Prediction

Dataset: Multiple trajectories of a F-16 (of length 1500) obtained from a 6 DoF Flight Dynamics Model

Practical Example: Aircraft Trajectory Prediction

Dataset: Each trajectory is comprised of:

- **Position**: Latitude, Longitude, Altitude -----> TARGET VARIABLES
- **Euler Angles**: Roll, Pitch, Yaw
- **Angular rates**: p, q, r
- **Linear Velocity:** u, v, w (and TAS)
- **Aerodynamic Angles:** alpha, beta
- **Linear Accelerations:** nx, ny, nz
- **Control actions:** ailerons, elevator, rudder, throttle

Acknowledgements

- MIT 6.S191: Recurrent Neural Networks
- Sequence Modeling and Recurrent Networks: J. Zico Kolter and Tianqi Chen. Carnegie Mellon University.
- Recurrent Networks Designing, Visualizing and Understanding Deep. Neural Networks: CS W182/282A Instructor: Sergey Levine. UC Berkeley.
- University of Cambridge. Recurrent Neural Networks.
- Andrej Karpathy's 2015 blog post.

UNIVERSIDAD
POLITÉCNICA
DE MADRID

UNIVERSIDAD
POLITÉCNICA
DE MADRID

ESCUELA TÉCNICA SUPERIOR
DE INGENIERÍA AERONÁUTICA
Y DEL ESPACIO